

CS253 Assignment 1

Memory Efficient Word Indexing System

Chinthala Vinayasri
Roll Number: 240310

1 Introduction

Processing very large log files is a common task in many real-world systems. However, loading an entire file into memory can lead to high memory usage and slow performance. The objective of this assignment is to design and implement a memory-efficient word indexing system using C++.

The program reads the input file using a fixed-size buffer and processes the file incrementally. Each word in the file is extracted and stored in a word index that records how many times the word appears in the dataset.

Words are defined as contiguous sequences of alphanumeric characters and are treated in a case-insensitive manner.

2 Program Design

The program is implemented using an object-oriented design with several classes that separate different responsibilities.

2.1 BufferedReader

The `BufferedReader` class reads the input file using a fixed-size buffer. Instead of loading the entire file into memory, the file is processed in chunks. This ensures that memory usage remains small even for large files.

2.2 Tokenizer

The `Tokenizer` class extracts words from the input text. Words are identified as sequences of alphanumeric characters. All extracted words are converted to lowercase so that the indexing process is case-insensitive.

2.3 WordIndex

The `WordIndex` class stores the frequency of each word using an `unordered_map`. Each unique word is stored as a key and the corresponding frequency is stored as the value.

2.4 Query Classes

An abstract base class named `Query` defines a common interface for executing queries. Three derived classes implement the required query functionality.

- `WordQuery` – computes the frequency of a specific word
- `TopQuery` – displays the top-K most frequent words
- `DiffQuery` – computes the difference in word frequency between two versions

This design demonstrates inheritance and runtime polymorphism.

3 Index Construction

During execution, the program reads the input file in fixed-size chunks. Each chunk is passed to the tokenizer, which extracts words from the text. The extracted words are then inserted into the word index where their frequencies are updated.

Special care is taken to correctly handle words that may be split across buffer boundaries. This ensures that words are not incorrectly divided when a buffer ends in the middle of a token.

4 Memory Usage

The program uses a fixed buffer size between 256 KB and 1024 KB. Only a small portion of the file is processed at a time.

Because the program only stores the frequency of unique words, the memory usage grows only with the number of distinct words rather than the size of the file. This allows the program to handle very large datasets efficiently.

5 Queries Supported

The system supports three types of queries.

5.1 Word Query

Returns the number of times a given word appears in the dataset.

```
./analyzer --file test_logs.txt --version v1 --buffer 512 --query  
word --word error
```

5.2 Top-K Query

Displays the K most frequent words in the dataset.

```
./analyzer --file test_logs.txt --version v1 --buffer 512 --query  
top --top 10
```

5.3 Difference Query

Computes the difference in frequency of a word between two versions.

```
./analyzer --file1 test_logs.txt
           --version1 v1
           --file2 verbose_logs.txt
           --version2 v2
           --buffer 512
           --query diff
           --word error
```

6 Example Output

Example output produced by the program:

```
Version: v1
Word: error
Count: 1452
Buffer size: 512 KB
Execution time: 0.02 sec
```

7 C++ Concepts Used

The implementation demonstrates several C++ programming concepts:

- Object-Oriented Programming
- Abstract base classes
- Inheritance
- Runtime polymorphism
- Function overloading
- Exception handling
- Template functions
- STL containers and algorithms

8 Conclusion

This assignment demonstrates how large text files can be processed efficiently using buffered file reading. The implemented system successfully builds a word-level index and supports multiple queries while maintaining low memory usage.

The object-oriented design improves modularity and makes the system easy to extend with additional queries in the future.